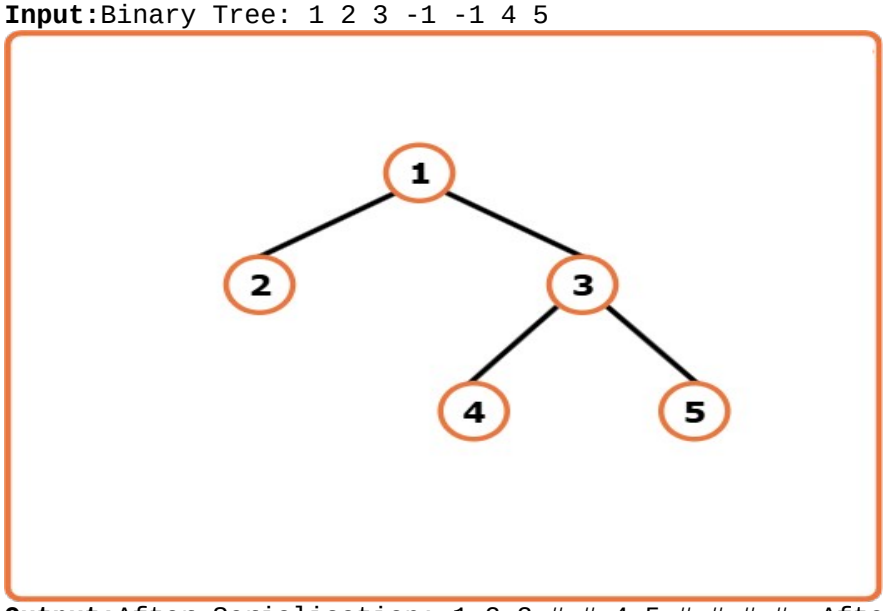
Serialize And Deserialize a Binary Tree

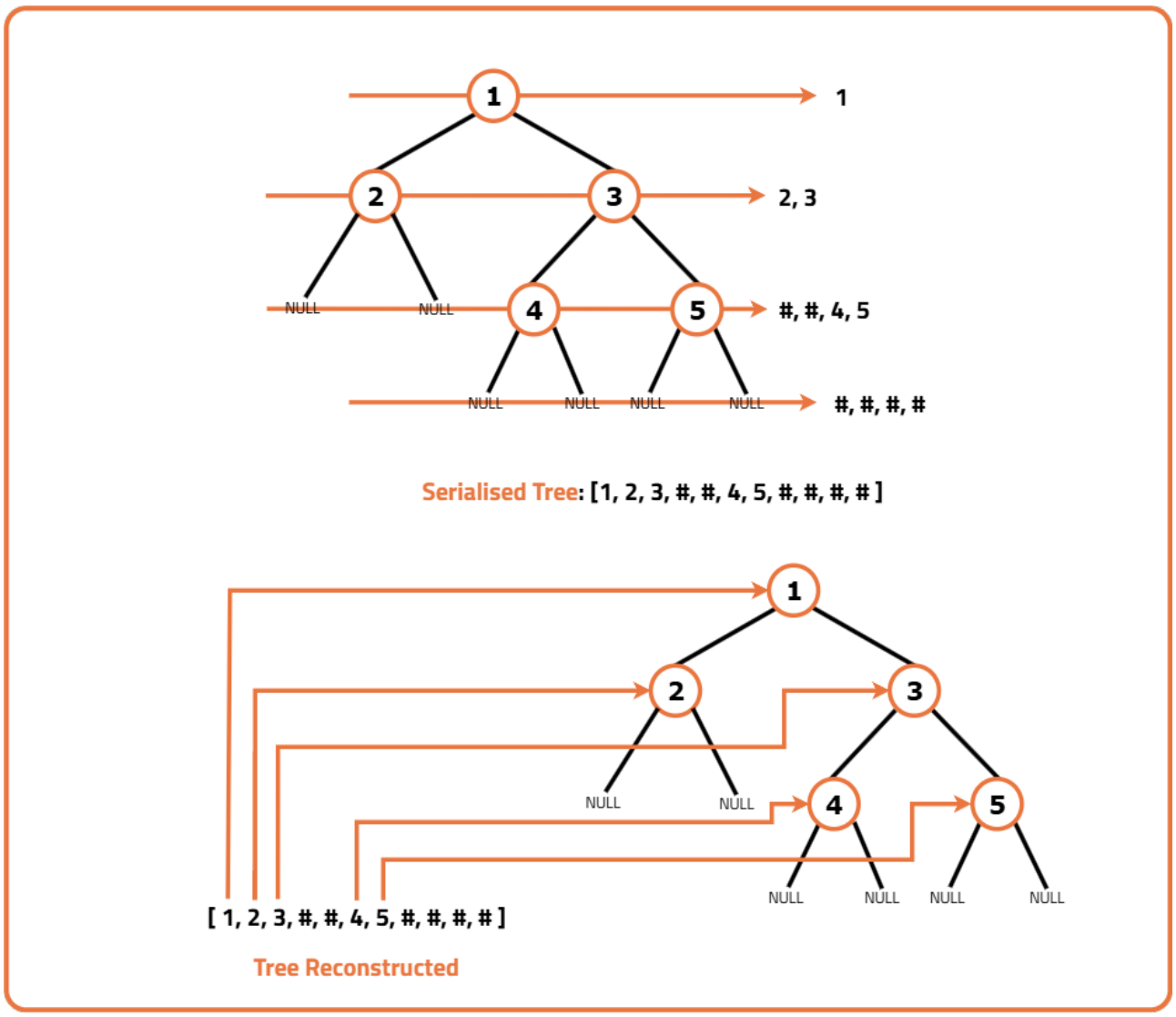
Problem Statement: Given a Binary Tree, design an algorithm to serialise and deserialise it. There is no restriction on how the serialisation and deserialization takes place. But it needs to be ensured that the serialised binary tree can be deserialized to the original tree structure. Serialisation is the process of translating a data structure or object state into a format that can be stored or transmitted (for example, across a computer network) and reconstructed later. The opposite operation, that is, extracting a data structure from stored information, is deserialization.

Examples

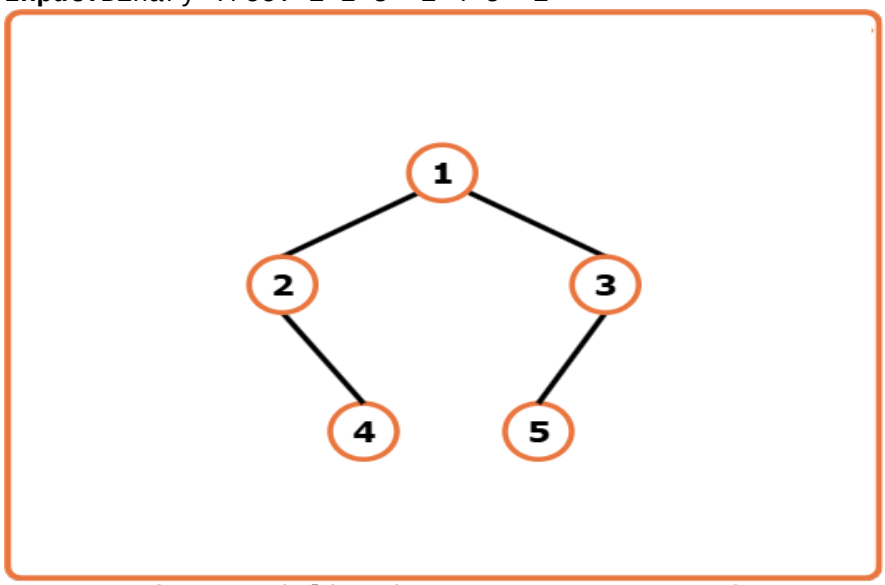
Input: Binary Tree: 1 2 3 -1 -1 4 5



Output: After Serialisation: 1,2,3,#,#,4,5,#,#,#,#, After Deserialization: (Original Tree Back)
Explanation: Any algorithm that compresses this binary tree to a string which can be transmitted and from which the binary tree can be reconstructed later can be used.
Here we have used a serialisation algorithm based on level order traversal where comma separates the nodes and # denotes null nodes.



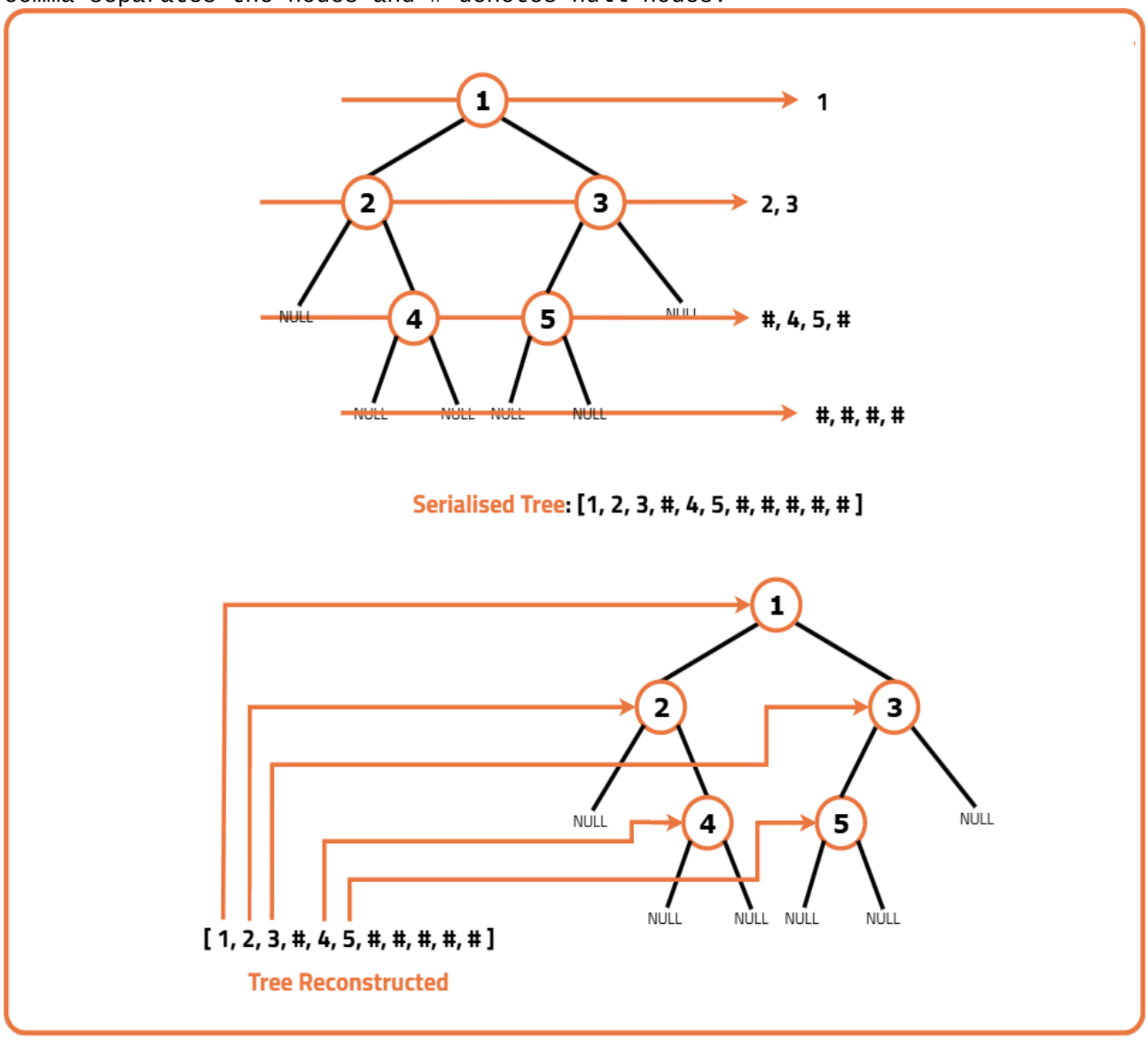
Input: Binary Tree: 1 2 3 -1 4 5 -1



Output : After Serialisation: 1,2,3,#,4,5,#, After Deserialization: (Original Tree Back)

Explanation: Any algorithm that compresses this binary tree to a string which can be transmitted and from which the binary tree can be reconstructed later can be used.

Here we have used a serialisation algorithm based on level order traversal where comma separates the nodes and # denotes null nodes.

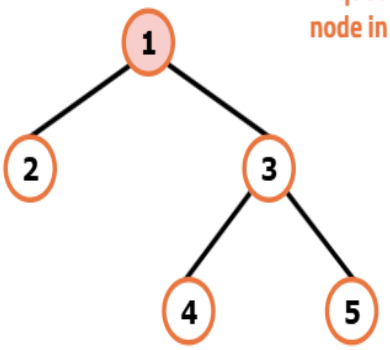


Approach
Algorithm

Serialisation:

- Check if the tree is empty: If the root is null, return an empty string.
- Initialise an empty string to store the serialised binary tree.
- Use a queue for level-order traversal: initialise a queue and enqueue the root.
- Within the level-order traversal loop:
    - Dequeue a node from the queue.
    - If the node is null, append "#" to the string.
    - If the node is not null, append its data value along with a comma (",") as a delimiter. Enqueue its left and right children.
- Return the final string containing the serialised representation of the tree.

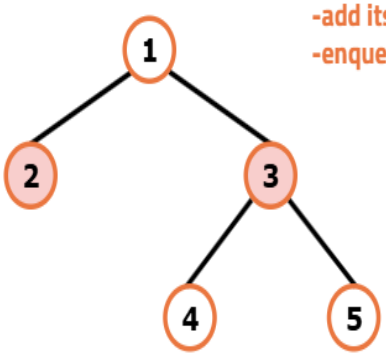
Enqueue the root
node in the queue



queue <TreeNode*>

serialised string = ""

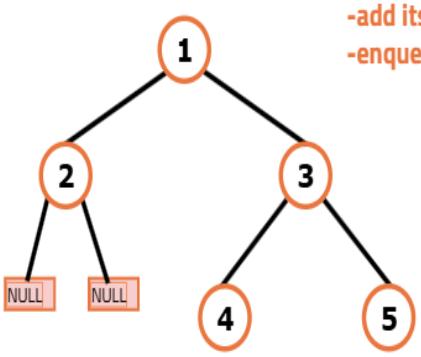
pop the front node
-add its value to the serialised string
-enqueue its children to the queue



queue <TreeNode*>

serialised string = "1,"

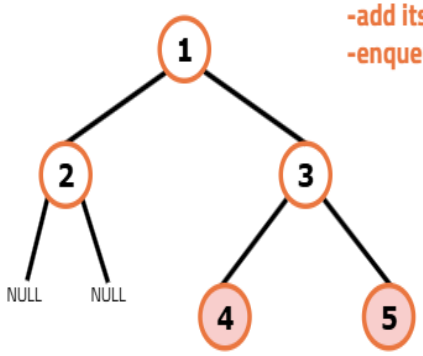
pop the front node
-add its value to the serialised string
-enqueue its children to the queue



queue <TreeNode*>

serialised string = "1,2,"

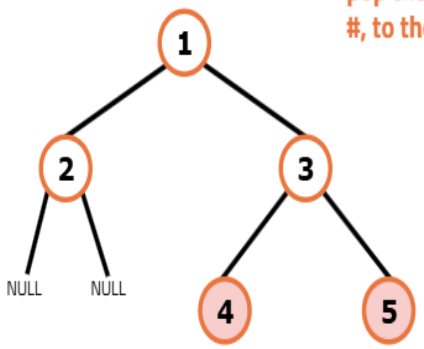
pop the front node
-add its value to the serialised string
-enqueue its children to the queue



queue <TreeNode*>

serialised string = "1,2,3,"

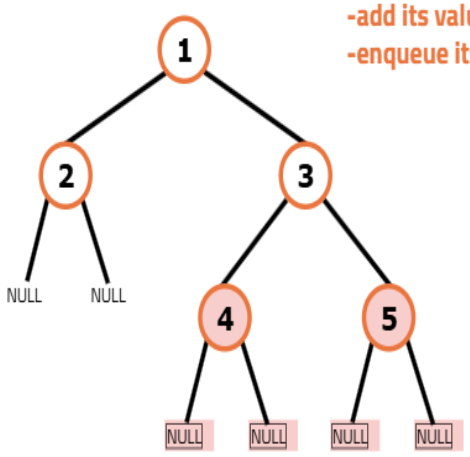
pop the front node, since its a null, we append
#, to the serialised string. (For both the null)



queue <TreeNode*>

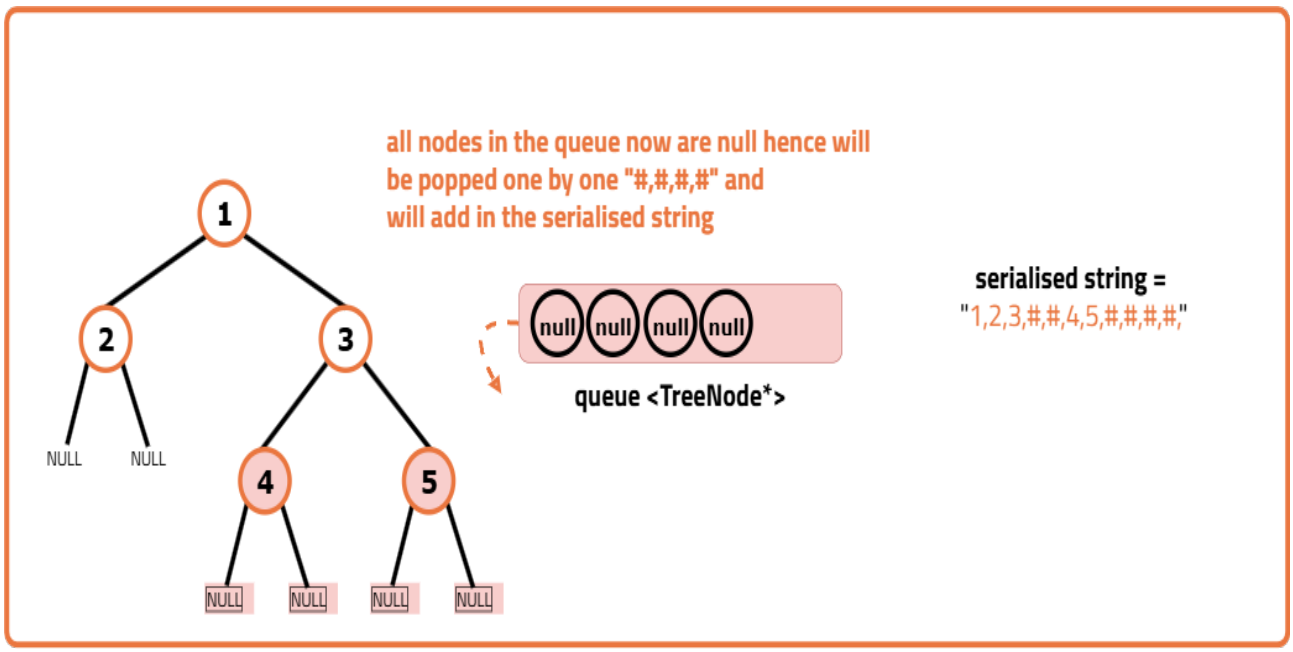
serialised string = "1,2,3,#,#"

pop the front node --> 4, 5
-add its value to the serialised string
-enqueue its children to the queue



queue <TreeNode*>

serialised string = "1,2,3,#,#,4,5"



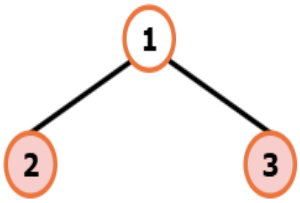
Deserialization:

- Check if the serialised data is empty: If it is, return null.
- Tokenize the serialised data using a stringstream with comma as a delimiter.
- Read the first token and create the root node with this value.
- Use a queue for level-order traversal: initialise a queue and enqueue the root.
- Within the level-order traversal loop:
    - Dequeue a node from the queue.
    - Read the value for the left child.
    - If it is "#", set the left child to null. Otherwise, create a new node and set it as the left child.
    - Read the next value for the right child.
    - If it is "#", set the right child to null. Otherwise, create a new node and set it as the right child.
    - Enqueue the left and right children into the queue for further traversal.
- Return the reconstructed root of the tree.

Read the first value of the stringstream
and create the root node with this value

queue <TreeNode*>

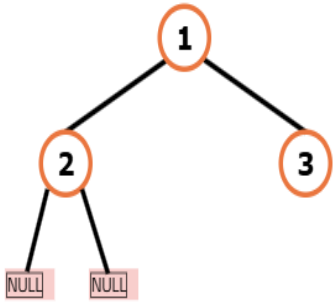
serialised string =
"1,2,3,#,#,4,5,#,#,#,#,"



Take the rest two values from stringstream and add
them as the left and right child of the popped node 2

queue <TreeNode*>

serialised string =
"1,2,3,#,#,4,5,#,#,#,#,"

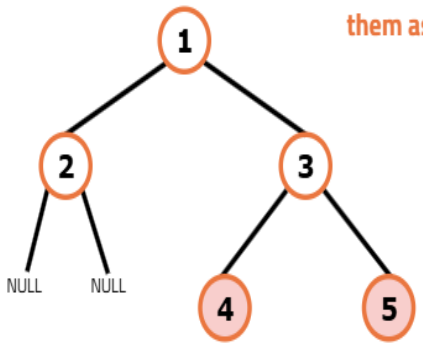


The next two values from the stringstream are #
hence we add null children to popped node 2

queue <TreeNode*>

serialised string =
"1,2,3,#,#,4,5,#,#,#,#,"

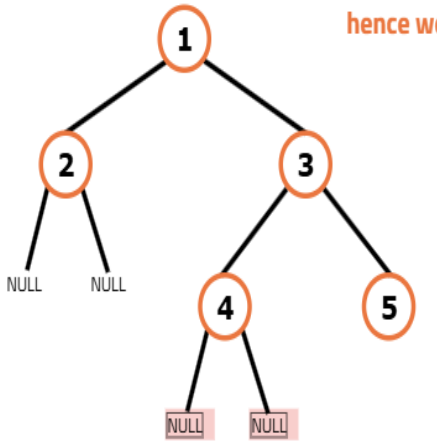
Take the rest two values from stringstream and add them as the left and right child of the popped node 3



queue <TreeNode*>

serialised string =
"1,2,3,#,#,4,5,#,#,#,#,"

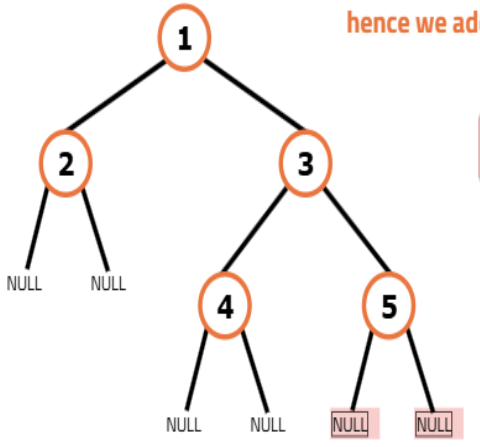
The next two values from the stringstream are # hence we add null children to popped node 4



queue <TreeNode*>

serialised string =
"1,2,3,#,#,4,5,#,#,#,#,"

The next two values from the stringstream are # hence we add null children to popped node 5



queue <TreeNode*>

serialised string =
"1,2,3,#,#,4,5,#,#,#,#,"

Code

```
import java.util.*;

// Definition for a binary tree node
class TreeNode {
    int val;
    TreeNode left;
    TreeNode right;

    // Constructor to initialize node value and child pointers
    TreeNode(int x) {
        val = x;
        left = null;
        right = null;
    }
}

class Solution {

    // Function to serialize a binary tree into a string using level-order
traversal
    public String serialize(TreeNode root) {

        // If tree is empty, return an empty string
        if (root == null) return "";

        // Initialize string to store serialized result
        StringBuilder s = new StringBuilder();

        // Initialize a queue to store nodes during level-order traversal
        Queue<TreeNode> q = new LinkedList<>();

        // Push root node into the queue
        q.offer(root);

        // Loop while queue is not empty
        while (!q.isEmpty()) {

            // Get the current node from the front of the queue
            TreeNode curNode = q.poll();

            // If current node is null, append "#" to string
            if (curNode == null) {
                s.append("#,");
            } else {
                // Append node value to string
                s.append(curNode.val).append(",");

                // Push left child into queue
                q.offer(curNode.left);

                // Push right child into queue
                q.offer(curNode.right);
            }
        }

        // Return the serialized tree string
        return s.toString();
    }

    // Function to deserialize a string and reconstruct the binary tree
    public TreeNode deserialize(String data) {
```

```
        // If data is empty, return null
        if (data.isEmpty()) return null;

        // Use string splitting to parse the input data
        String[] values = data.split(",");

        // Create the root node
        TreeNode root = new TreeNode(Integer.parseInt(values[0]));

        // Initialize a queue to hold tree nodes for level-order reconstruction
        Queue<TreeNode> q = new LinkedList<>();

        // Push root node into the queue
        q.offer(root);

        // Index for navigating the value array
        int i = 1;

        // Loop through the array to construct the tree
        while (!q.isEmpty() && i < values.length) {

            // Get the current node from the front of the queue
            TreeNode node = q.poll();

            // Read the left child value
            if (!values[i].equals("#")) {
                TreeNode leftNode = new TreeNode(Integer.parseInt(values[i]));
                node.left = leftNode;
                q.offer(leftNode);
            }
            i++;

            // Read the right child value
            if (!values[i].equals("#")) {
                TreeNode rightNode = new TreeNode(Integer.parseInt(values[i]));
                node.right = rightNode;
                q.offer(rightNode);
            }
            i++;
        }

        // Return the root of the reconstructed tree
        return root;
    }

    // Function to perform in-order traversal and print the tree
    public void inorder(TreeNode root) {
        if (root == null) return;
        inorder(root.left);
        System.out.print(root.val + " ");
        inorder(root.right);
    }
}

// Driver code
class Main {
    public static void main(String[] args) {

        // Manually construct the binary tree
        TreeNode root = new TreeNode(1);
        root.left = new TreeNode(2);
        root.right = new TreeNode(3);
        root.right.left = new TreeNode(4);
```

```
        root.right.right = new TreeNode(5);

        // Create an instance of the solution class
        Solution solution = new Solution();

        // Print original tree using in-order traversal
        System.out.print("Orignal Tree: ");
        solution.inorder(root);
        System.out.println();

        // Serialize the tree into a string
        String serialized = solution.serialize(root);
        System.out.println("Serialized: " + serialized);

        // Deserialize the string back into a tree
        TreeNode deserialized = solution.deserialize(serialized);

        // Print tree after deserialization
        System.out.print("Tree after deserialisation: ");
        solution.inorder(deserialized);
        System.out.println();
    }
}
```

Complexity Analysis

Time Complexity: O(N)

1. serialize function: O(N), where N is the number of nodes in the tree. This is because the function performs a level-order traversal of the tree, visiting each node once.
2. deserialize function: O(N), where N is the number of nodes in the tree. Similar to the serialize function, it processes each node once while reconstructing the tree.

Space Complexity: O(N)

1. serialize function: O(N), where N is the maximum number of nodes at any level in the tree. In the worst case, the queue can hold all nodes at the last level of the tree.
2. deserialize function: O(N), where N is the maximum number of nodes at any level in the tree. The queue is used to store nodes during the reconstruction process, and in the worst case, it may hold all nodes at the last level.